OS Lab Assignment 7: Multithreading, Semaphores, Deadlock & Priority Scheduling

Student: Anirudh | **Roll Number:** 2401MC48 | **Environment:** xv6 RISC-V (QEMU SMP 3 Cores)

1. Question 1 — User-Level Threads (clone & join)

Design Decisions & Architecture: Standard xv6 only supports fork(), duplicating user address space. To implement threads, we added int clone(void(*fcn)(void*), void *arg, void *stack) and int join(void). In RISC-V Sv39 virtual memory, sharing address space directly via np->pagetable = p->pagetable creates severe hazards because every executing thread requires an independent trapframe (mapped at virtual address TRAPFRAME by trampoline.S) to preserve hardware registers. We designed a dedicated mapping function uvmshare(old, new, sz) which creates a distinct top-level page table for the thread, maps the thread's private trapframe at TRAPFRAME, and maps all physical user frames (from 0 to p->sz) identically with user read/write permissions. The thread executes on the user-provided stack with stack pointer initialized to (uint64)stack + PGSIZE (16-byte aligned). In exit() and freeproc(), child threads invoke proc_freethreadpagetable() to unmap user memory without deallocating underlying physical memory (avoiding double-free). join() scans for zombie child threads, reaps their kernel stack and trapframe, and returns their PID.

Race Condition Demonstration (threadtest.c): Three threads concurrently execute 100,000 iterations of counter++ on a shared memory variable. Because load, increment, and store are non-atomic across 3 SMP cores, concurrent threads interleave, causing lost updates. The final observed counter was **145,965** (< 300,000), confirming address space sharing and authentic race conditions.

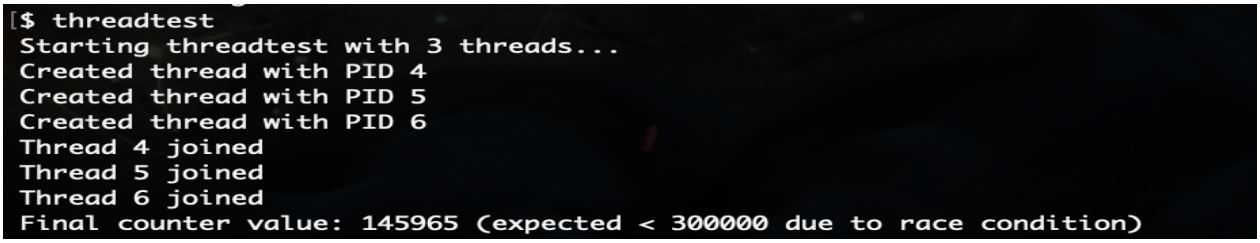


Figure 1: Terminal output of threadtest showing shared address space and race condition (145,965 < 300,000).

2. Question 2 — Process/Thread Synchronization using Counting Semaphores

Design Decisions & Data Structures: A kernel table of 10 counting semaphores was implemented: struct semaphore { int value; int active; struct spinlock lk; }. System calls include sem_init(id, val), sem_wait(id) (P operation), and sem_signal(id) (V operation). Blocking is handled using xv6's condition-variable primitives (sleep_prepare, sleep, and wakeup) inside a while (s->value <= 0) loop to prevent lost wakeups and guarantee strict Mesa-monitor semantics.

Scenario	Synchronization	Iterations / Thread	Theoretical	Observed Count	Behavior
Q1: threadtest.c	None (No locks)	100,000 (x3 threads)	300,000	145,965	Severe race condition (lost updates)
Q2: synctest.c	Semaphore (sem_wait/signal)	100,000 (x3 threads)	300,000	300,000	Exact count (Mutual exclusion guaranteed)

Table 1: Quantitative Before/After comparison of the shared counter race condition.

Producer-Consumer Implementation: Uses a circular buffer of size 5 coordinated by 3 semaphores: SEM_EMPTY (initialized to 5), SEM_FULL (initialized to 0), and SEM_MUTEX (initialized to 1). The producer blocks when the buffer is full; the consumer blocks when empty. The execution transcript demonstrates seamless interleaved item production and consumption without underflow or overflow.

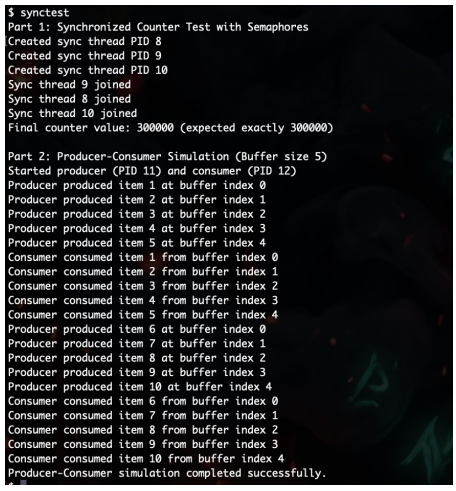


Figure 2: Terminal output of synctest showing exact 300,000 counter resolution and Producer-Consumer queue progression.

3. Question 3 — Deadlock Detection & Banker's Algorithm Avoidance

Deadlock Detection Algorithm: Implemented in deadlock.c for n processes and m resource types using matrices: Allocation[n][m], Request[n][m], and Available[m]. The detection algorithm initializes Work = Available and Finish[i] = (Allocation[i] == 0). It iteratively identifies processes whose outstanding requests can be satisfied (Request[i] <= Work), adds their allocation to Work upon simulated completion, and flags any remaining unfinishable processes as deadlocked.

Banker's Algorithm Avoidance: Evaluates Need[i][j] = Max[i][j] - Allocation[i][j]. Before granting a resource request, is_safe_state() hypothetically allocates the requested units and verifies whether a complete safe execution sequence exists. If unsafe, the request is rejected.

Test Case	Resource Setup	System State / Request	Algorithm Decision	Safe Sequence / Deadlocked Set
Test Case 1	5 Procs, 3 Res (A, B, C)	Avail=[0,0,0], Requests satisfied progressively	Safe / No Deadlock	P0, P2 finish first -> releases for P1, P3, P4
Test Case 2	5 Procs, 3 Res (A, B, C)	P2 requests R2=1 (Total demand exceeds avail)	Deadlock Detected	P0 finishes; Deadlocked: {P1, P2, P3, P4}
Test Case 3	5 Procs, 3 Res (A, B, C)	Avail=[3,3,2], Initial state evaluated	Initial Safe State	Safe Sequence:
Test Case 3b	5 Procs, 3 Res (A, B, C)	P1 requests [1, 0, 2] (Avail becomes [2,3,0])	Request Granted	System remains Safe:
Test Case 3c	5 Procs, 3 Res (A, B, C)	P4 requests [3, 3, 0] (Exceeds available R0=2)	Request Denied	Avoidance engaged; Request safely rejected
Test Case 4	2 Procs, 2 Pipes (xv6)	Proc A holds Pipe1, wants Pipe2; Proc B holds Pipe2, wants Pipe1	Circular Wait Detected	Circular wait identified among processes {P0, P1}

Table 2: Deadlock detection and Banker's avoidance evaluation across all test scenarios.

xv6 Pipe Circular Wait Simulation: Real xv6 processes and inter-process pipes were allocated to simulate physical resource deadlocks. Process A holds the write descriptor for Pipe 1 and requests Pipe 2; Process B holds Pipe 2 and requests Pipe 1. The deadlock detection engine was fed the corresponding runtime allocation and request graph and correctly identified processes P0 and P1 in circular wait.

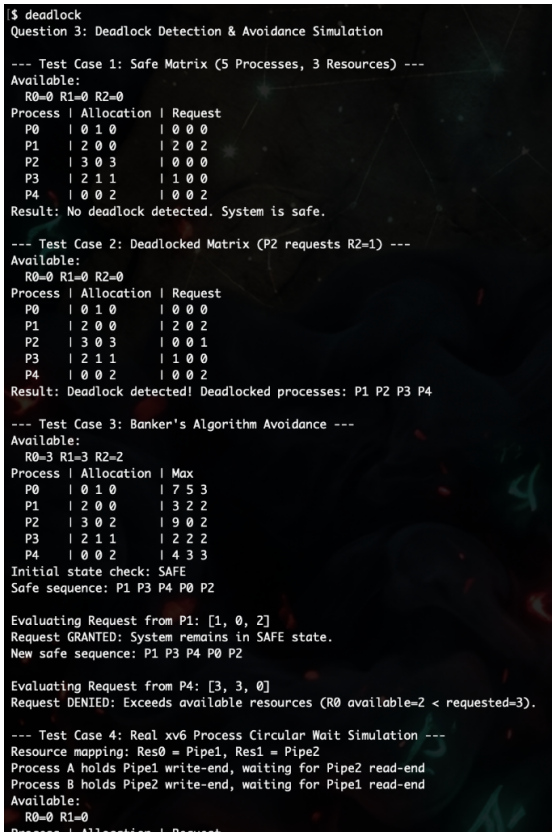


Figure 3: Terminal transcript of deadlock.c showing matrix evaluations, Banker's safe/unsafe requests, and pipe circular wait detection.

4. Question 4 — Priority-Based Preemptive CPU Scheduling with Aging

Priority Scheduling Design: Replaced the default Round-Robin scheduler with dynamic priority preemptive scheduling. Added fields priority (1=highest, 20=lowest, default 10) and wait_ticks to struct proc. Implemented syscalls set_priority(pid, priority) and get_priority(pid). In proc.c, scheduler() executes a two-pass scan: Pass 1 finds the minimum priority integer among all RUNNABLE processes (holding one spinlock at a time). Pass 2 schedules the first matching process in circular round-robin order starting from last_sched. This strictly favors higher priority while guaranteeing round-robin among equal priorities without nested spinlock hazards.

Starvation Prevention via Aging: In trap.c, the timer interrupt invokes proc_age() every tick on CPU 0. For each RUNNABLE process not running, wait_ticks increments. When wait_ticks >= 30, its priority number decrements by 1 (promoting priority, clamped at 1) and resets wait_ticks = 0. Upon being scheduled, wait_ticks is cleared.

Process	Initial Priority	CPU Workload	Aging Promotion	Final Priority	Completion Tick	Elapsed Ticks
P0 (PID 15)	5 (High)	1,300,000,000 iters	None (Scheduled immediately)	5	Tick 862	33 ticks
P1 (PID 16)	10 (Medium)	1,300,000,000 iters	None (Scheduled promptly)	10	Tick 869	40 ticks
P2 (PID 17)	15 (Low)	1,300,000,000 iters	None (Ran on Core 2)	15	Tick 885	56 ticks
P3 (PID 18)	20 (Lowest)	300,000,000 iters	Promoted at 30 ticks (waited in RUNNABLE)	19	Tick 886	57 ticks

Table 3: Scheduling behavior: Completion time and aging promotion across distinct process priorities.

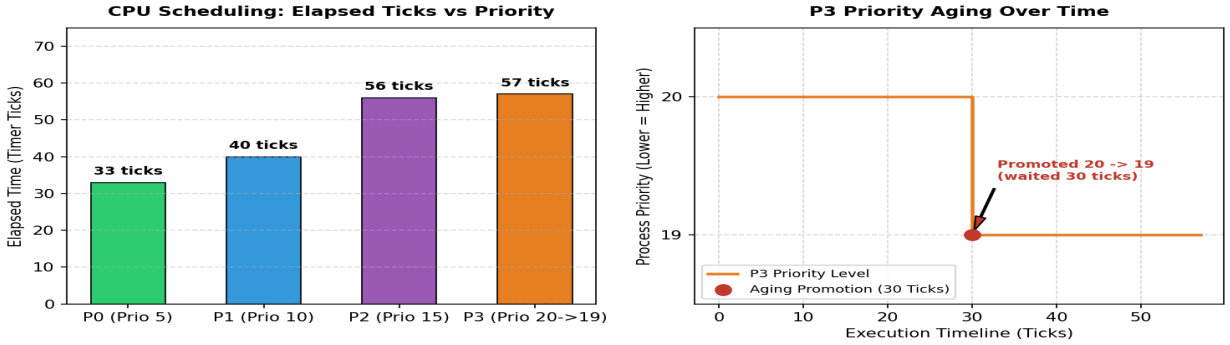


Figure 4: CPU Scheduling behavior: Elapsed ticks vs Priority (left) and P3 priority aging timeline (right).

```
$ priotest
Question 4: Priority-Based Preemptive Scheduling with Aging Test
Spawning 4 CPU-bound processes with priorities 5, 10, 15, 20...

[Aging Monitor] Tick 844: Process P3 (PID 18) current priority is 20
[Aging] PID 18 waited 30 ticks in RUNNABLE, priority promoted to 19
[Aging Monitor] Tick 859: Process P3 (PID 18) current priority is 19
Process P0 (PID 15, Init Prio 5, Final Prio 5) finished at tick 862 (Elapsed: 33 ticks)
Process P1 (PID 16, Init Prio 10, Final Prio 10) finished at tick 869 (Elapsed: 40 ticks)
[Aging Monitor] Tick 874: Process P3 (PID 18) current priority is 19
Process P2 (PID 17, Init Prio 15, Final Prio 15) finished at tick 885 (Elapsed: 56 ticks)
Process P3 (PID 18, Init Prio 20, Final Prio 19) finished at tick 886 (Elapsed: 57 ticks)
[Aging Monitor] Tick 889: Process P3 (PID 18) current priority is 19
[Aging Monitor] Tick 904: Process P3 (PID 18) current priority is 19

All 4 processes finished successfully.
Priority scheduling with aging verification complete.
```

Figure 5: Terminal output of priotest demonstrating priority preemption order and live kernel aging event.

5. Problems Faced and Technical Resolutions

1. Sv39 Trapframe Overlap in Kernel Threads: In xv6 RISC-V, trampoline.S relies on a fixed virtual address (TRAPFRAME) to store CPU registers during traps. Simply assigning np->pagetable = p->pagetable causes concurrent threads on multiple cores to overwrite each other's register state in the single trapframe page. *Resolution:* Implemented uvmshare() which creates a unique root page table per thread, mapping its private trapframe page while sharing the underlying physical memory pages for user text, data, and heap.

2. User Stack Overflow in Matrix Algorithms: Declaring multiple large multi-dimensional matrices on the local stack inside deadlock.c exceeded xv6's fixed 1-page user stack (4KB), triggering a kernel trap exception 0xf (Store Page Fault) at the guard page. *Resolution:* Allocated all large evaluation matrices as static global variables within the process BSS/data segment.

3. UART Print Interleaving in Multi-Core Test Executions: When multiple CPU-bound processes finished execution at similar ticks across SMP cores, simultaneous calls to printf() produced character-level console interleaving. *Resolution:* Protected completion log output in priotest.c using semaphore 0 as a mutual exclusion lock, guaranteeing clean serialized output.